

Cyber Security

Password Authentication

By
Dr. Mudassar Raza
Professor
Department of Computer Science
Namal University Mianwali



https://techgirls.ece.vt.edu/slides/introduction_to_cybersecurity.html#39

User Authentication

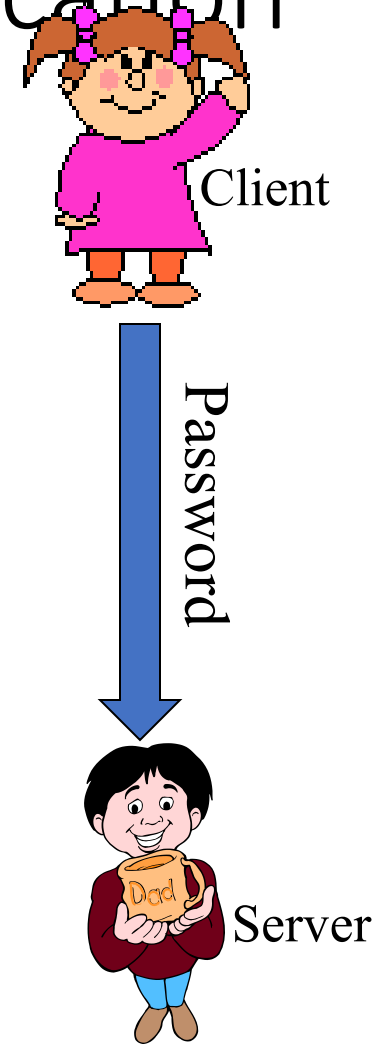
- Using a method to validate users who attempt to access a computer system or resources, to ensure they are authorized
- Types of user authentication
 - **Something you know**
 - E.g., user account names and passwords
 - **Something you have**
 - Smart cards or other security tokens
 - **Something you are**
 - Biometrics

Variants of Passwords

- Password
- Passphrase
 - a sequence of words or other text used for similar purpose as password
- Passcode
- Personal identification number (PIN)

Scenarios Requiring User Authentication

- Scenarios
 - Logging into a local computer
 - Logging into a computer remotely
 - Logging into a network
 - Access web sites
- Vulnerabilities can exist at client side, server side, or communications channel.



Threats to Passwords

- Eavesdropping (insecure channel between client and server)
- Login spoofing (human errors), shoulder surfing, keyloggers
- Offline dictionary attacks
- Social engineering (human errors)
 - e.g., pretexting: creating and using an invented scenario (the pretext) to persuade a target to release information or perform an action and is usually done over the telephone
- Online guessing (weak passwords)

Guessing Attacks: Two Factors for Password Strength

- The average number of guesses the attacker must make to find the correct password
 - determined by how unpredictable the password is, including how long the password is, what set of symbols it is drawn from, and how it is created.
- The ease with which an attacker can check the validity of a guessed password
 - determined by how the password is stored, how the checking is done, and any limitation on trying passwords

Password Entropy: The Value of Unpredictable Passwords

- Password entropy is a measurement of how unpredictable, and therefore un-guessable, a password is.
- Most of us don't have passwords that pass the entropy test. For example, "123456" and "qwerty" were two of the top passwords used in 2021.
- Aka. Guess entropy

Password Entropy: The Value of Unpredictable Passwords

- Password entropy is a measurement of difficulty. A hacker might try:
 - **Simple guesses.** The hacker uses well-known passwords like “password” to unlock access.
 - **Brute force.** The hacker uses a program that submits dozens of guesses every minute based on mathematical probabilities.
 - **Research.** The hacker digs through your social media accounts and online presence to find out your street address, pet’s name, and other common data points used in passwords.

Password entropy measures the likelihood that these hacking methods will work and the intruder gets in.

Try a password entropy calculator

- Password entropy is typically expressed in bits. Low scores indicate passwords that are very easy to crack. The higher you rank, the harder a hacker must work.
- A password entropy calculator opens in a new tab equation looks like this:

$$E = \log_2(R^L)$$

- E stands for password entropy.
- R stands for possible characters within the password.
- L stands for the number of characters in your password.

Try a password entropy calculator

Calculate password entropy

- **Formula:** $E = \log_2(R^L)$ or $E = L \times \log_2(R)$
- **E :** Entropy, measured in bits
- **R :** The number of possible characters in your password's character set
 - Lowercase letters (a-z): 26
 - Uppercase letters (A-Z): 26
 - Numbers (0-9): 10
 - Symbols (e.g., !, @, #, \$): Approximately 32 for a common set
 - Total for a mix of uppercase, lowercase, numbers, and symbols is approximately 95
- **L :** The number of characters in the password 

Example

- **Password:** P@\$\$w0rd
- **Length (L):** 8
- **Character set (R):** Uppercase, lowercase, numbers, symbols = 95
- **Calculation:** $E = 8 \times \log_2(95)$
 - $\log_2(95) \approx 6.58$
 - $E \approx 8 \times 6.58 \approx 52.64$ bits

Example of Weak Passwords (from Wikipedia)

- **Default passwords** (as supplied by the system vendor and meant to be changed at installation time): *password, default, admin, guest*, etc.
- Dictionary words: *chameleon, RedSox, sandbags, bunnyhop!, IntenseCrabtree*, etc.
- **Words with numbers appended**: *password1, deer2000, john1234*, etc.,
- **Words with simple obfuscation**: *p@ssw0rd, l33th4x0r, g0ldf1sh*, etc.
- **Doubled words**: *crabcrab, stopstop, treetree, passpass*, etc., can be easily tested automatically.

Example of Weak Passwords (from Wikipedia)

- **Common sequences from a keyboard row:** *qwerty, 12345, asdfgh, fred*, etc.
- **Numeric sequences** based on well known numbers such as 911, 314159, or 27182, etc.,
- **Identifiers:** *jsmith123, 1/1/1970, 555-1234, "your username"*, etc.,
- **Anything personally related to an individual:** license plate number, Social Security number, current or past telephone number, student ID, address, birthday, sports team, relative's or pet's names/nicknames/birthdays, etc.,
 - can easily be tested automatically after a simple investigation of person's details.

Top 20 Passwords In 32 million RockYou.com passwords

<http://www.tomshardware.com/news/imperva-rockyou-most-common-passwords,9486.html>

The Top 500 Worst Passwords of All Time

<http://www.whatsmypass.com/the-top-500-worst-passwords-of-all-time>

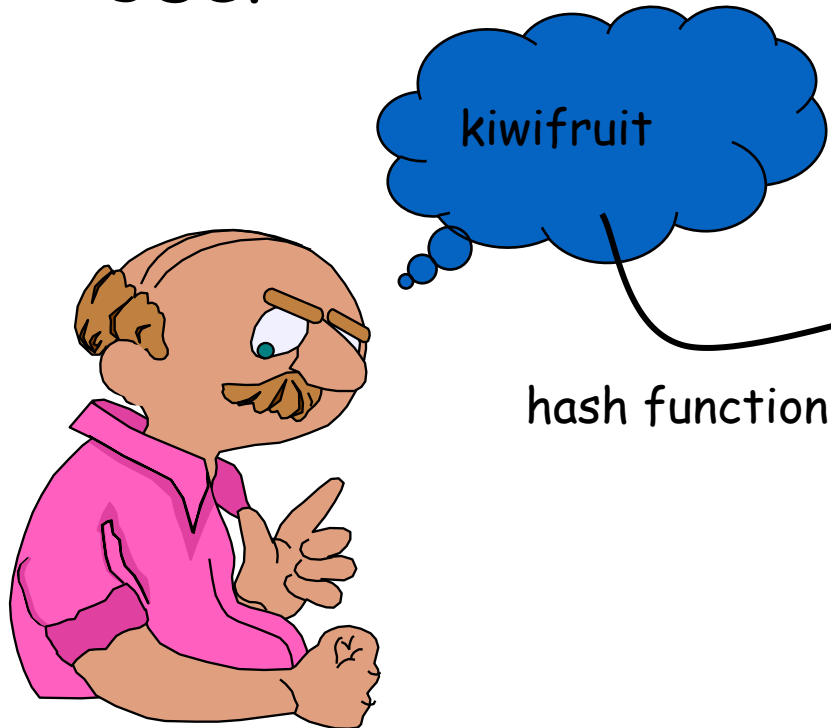
Mechanisms to Avoid Weak Passwords

- Allow long passphrases, forbid short passwords
- Randomly generate passwords where appropriate
 - Though probably inappropriate for most scenarios
- Give user suggestions/guidelines in choosing passwords
 - e.g., think of a sentence and select letters from it, “It’s 12 noon and I am hungry” => “I’S12&IAH”
 - Using both letter, numbers, and special characters
- Check the quality of user-selected passwords
 - Use a number of rules of thumb; run dictionary attack tools
 - Evaluate strength of a password and explain the weaknesses
- This is an open research question
 - Any suggestion?

Balancing Password Entropy & Usability Concerns

- Forcing randomly generated passwords is often bad.
 - A user needs to remember passwords for tens, if not hundreds of accounts
 - High entropy passwords are difficult to remember
- The “Weakest Link” security principle applies here
 - Often times, guessing passwords is not the weakest link
 - One can use various ways to reduce adversary’s abilities to test password guesses
 - When a user cannot remember the password for an account, there must be a way to allow a user to retrieve it.
 - The recovering method either has low security, or costs lots of money
 - It creates a weaker link.

User



Password file

exrygbzyf
kgnosfix
ggjoklbsz
...
...

Basic password authentication

- Setup
 - User chooses password
 - Hash of password stored in password file
- Authentication
 - User logs into system, supplies password
 - System computes hash, compares to file
- Attacks
 - Online dictionary attack
 - Guess passwords and try to log in
 - Offline dictionary attack
 - Steal password file, try to find p with $\text{hash}(p)$ in file

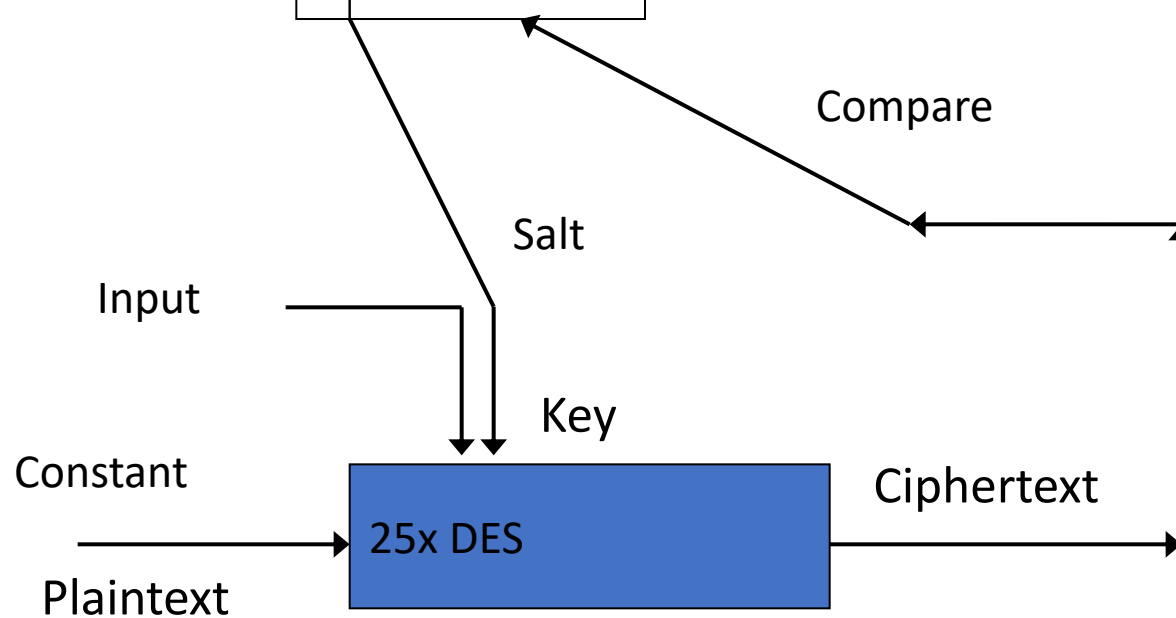
Dictionary Attack – some numbers

- Typical password dictionary
 - 1,000,000 entries of common passwords
 - people's names, common pet names, and ordinary words.
 - Suppose you generate and analyze 10 guesses per second
 - This may be reasonable for a web site; offline is *much* faster
 - Dictionary attack in at most 100,000 seconds = 28 hours, or 14 hours on average
- If passwords were random
 - Assume six-character password
 - Upper- and lowercase letters, digits, 32 punctuation characters
 - 689,869,781,056 password combinations.
 - Exhaustive search requires 1,093 years on average

Salt

- Unix password line

walt:fURfuu4.4hY0U:129:129:Belgers:/home/walt:/bin/csh



When password is set, salt is chosen randomly

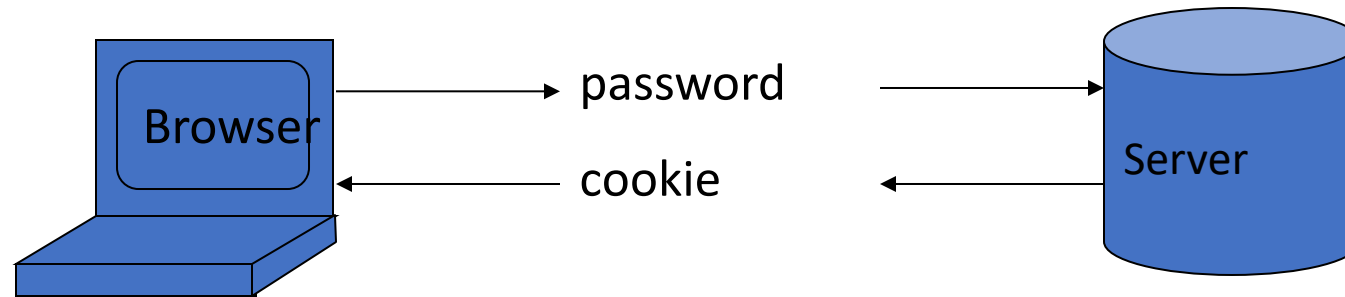
Advantages of salt

- Without salt
 - Same hash functions on all machines
 - Compute hash of all common strings once
 - Compare hash file with all known password files
- With salt
 - One password hashed 2^{12} different ways
 - Precompute hash file?
 - Need much larger file to cover all common strings
 - Dictionary attack on known password file
 - For each salt found in file, try all common strings

Mechanisms to Defend Against Dictionary and Guessing Attacks

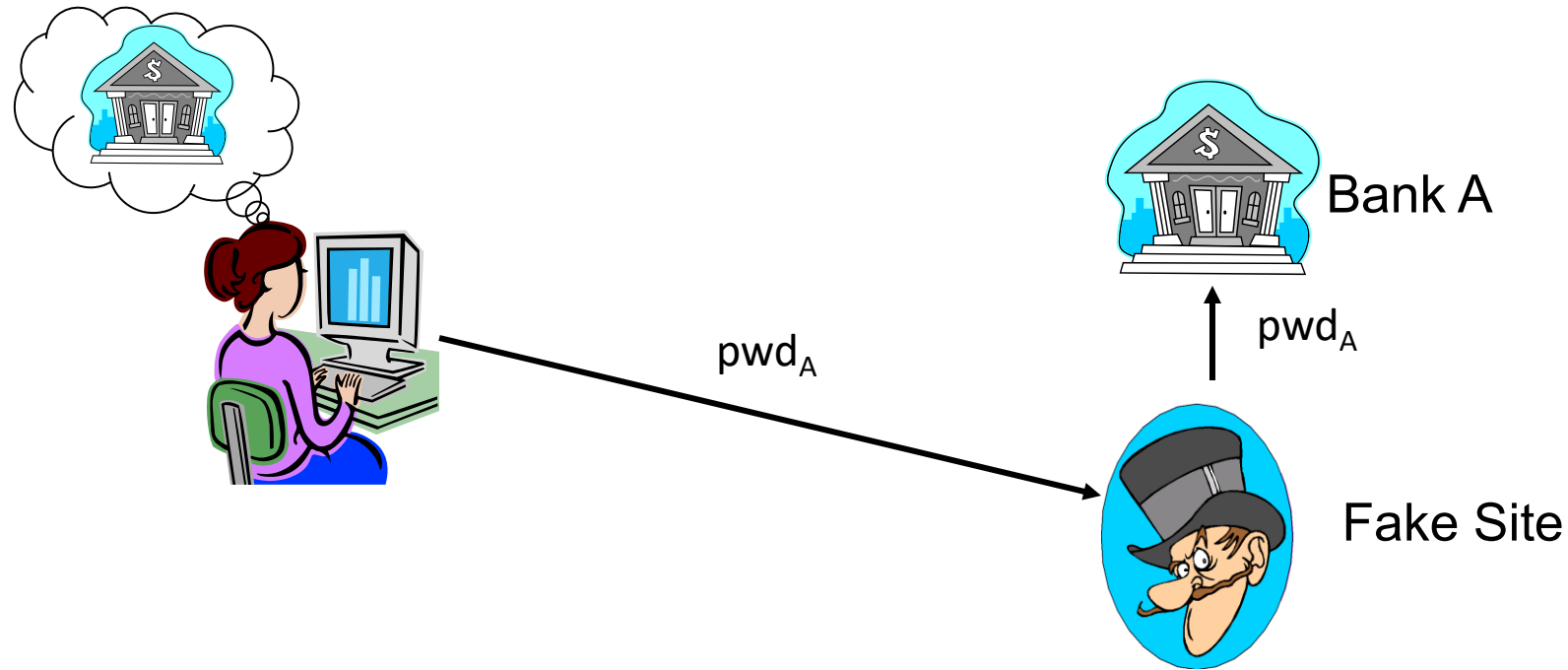
- Protect stored passwords (use both cryptography & access control)
 - An instance of the “Defense in Depth” principle:
 - Use multiple independent method of defense, so that even if one layer fails, security is still not compromised
 - Consider, e.g., password dataset compromises
- Disable accounts with multiple failed attempts
- Require extra authentication mechanism (e.g., phone, other email account, etc.)

Web Authentication



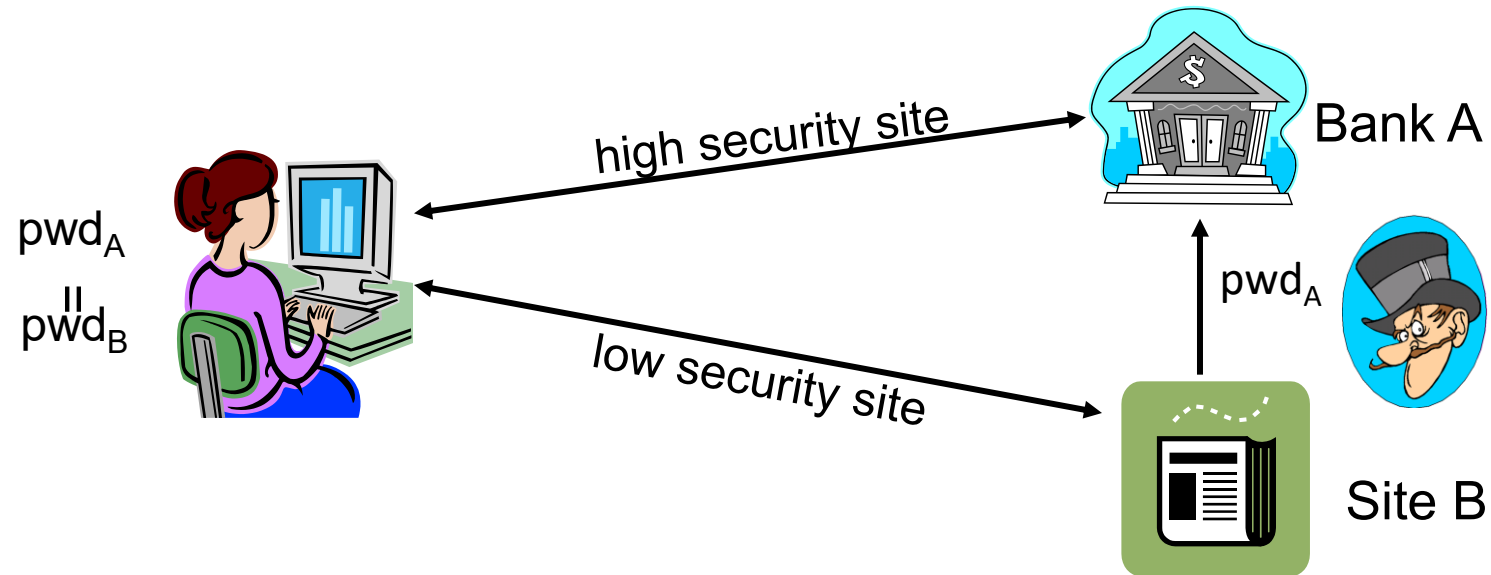
- Problems
 - Network sniffing
 - Malicious or weak-security website
 - Phishing
 - Common password problem
 - Pharming – DNS compromise
 - Malware on client machine
 - Spyware
 - Session hijacking, fabricated transactions
- } next few slides

Password Phishing Problem



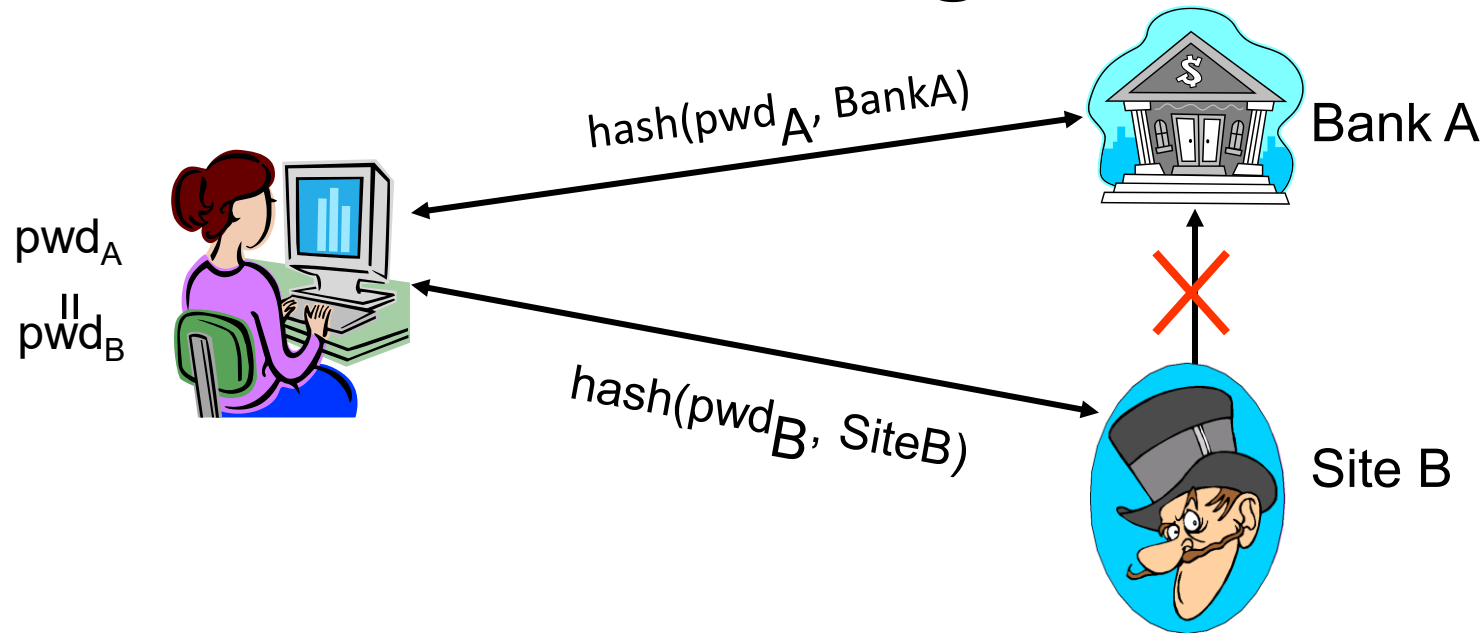
- User cannot reliably identify fake sites
- Captured password can be used at target site

Common Password Problem



- Phishing attack or break-in at site B reveals pwd_A
 - Server-side solutions will not keep pwd safe
 - Solution: Strengthen with client-side support

Defense: Password Hashing



- Generate a unique password per site
 - $\text{HMAC}_{\text{fido:123}}(\text{banka.com}) \Rightarrow \text{Q7a+0ekEXb}$
 - $\text{HMAC}_{\text{fido:123}}(\text{siteb.com}) \Rightarrow \text{OzX2+ICiqc}$
- Hashed password is not usable at any other site
 - Protects against password phishing
 - Protects against common password problem

Mechanisms to Defend Against Login Spoofing: Trusted Path

- Login Spoofing Attacks:
 - write a program showing a login window on screen and record the passwords
 - put su in current directory
- Defense: Trusted Path
 - Mechanism that provides confidence that the user is communicating with the real intended server
 - attackers can't intercept or modify whatever information is being communicated.
 - defends attacks such as fake login programs
 - Example: Ctrl+Alt+Del for log in on Windows
 - Causes a non-maskable interrupt that can only be intercepted by the operating system, guaranteeing that the login window cannot be spoofed

Spoofing & Defenses on the Web

- Phishing attacks
 - attempting to acquire sensitive information such as usernames, passwords and credit card details by masquerading as a trustworthy entity in electronic communication.
- Website forgery
 - Set up fake websites that look like e-commerce sites and trick users into visiting the sites and entering sensitive info
- Defense methods
 - Browser filtering of known phishing sites
 - Cryptographic authentication of servers (will talk about in future)
 - User-configured authentication of servers
 - To ensure that the site is the one the human user has in mind
 - E.g., site key, pre-selected picture/phrases

KeyLogging

- Threats from insecure client side
- Keystroke logging (keylogging) is the action of tracking (or logging) the keys struck on a keyboard, typically in a covert manner so that the person using the keyboard is unaware that their actions are being monitored.
- Software -based
 - key-stroke events, grab web forms, analyze HTTP packets
- Hardware-based
 - Connector, wireless sniffers, acoustic based
- Defenses:
 - Anti-spyware, network monitors, on-screen soft keyboard, automatic form filler, etc.
- In general difficult to deal with once on the system



Using Passwords Over Insecure Channels

- One-time passwords
 - Each password is used only once
 - Defend against passive adversaries who eavesdrop and later attempt to impersonate
- Challenge response
 - Send a response related to both the password and a challenge
- Zero knowledge proof of knowledge
 - Prove knowledge of a secret value, without leaking any info about the secret

How to do One-Time Password

- Shared lists of one-time passwords
- Time-synchronized OTP
 - E.g., use $MAC_K(t)$, where K is shared secret, and t is current time
- Using a hash chain (Lamport)
 - $h(s), h(h(s), h(h(h(s)))), \dots, h^{1000}(s)$
 - use these values as passwords in reverse order



Lamport's One-Time Password: Using a Hash Chain

- One-time setup:
 - A selects a value w , a hash function $H()$, and an integer t , computes $w_0 = H^t(w)$ and sends w_0 to B
 - B stores w_0
- Protocol: to identify to B for the i^{th} time, $1 \leq i \leq t$
 - A sends to B: $A, i, w_i = H^{t-i}(w)$
 - B checks $i = i_A, H(w_i) = w_{i-1}$
 - if both holds, $i_A = i_A + 1$

Challenge-Response Protocols

- Goal: one entity authenticates to other entity proving the knowledge of a secret, 'challenge'
- How to design this using the crypto tool we have learned?
- Approach: Use time-variant parameters to prevent replay, interleaving attacks, provide uniqueness, and timeliness
 - e.g., nonce (used only once), timestamps

Challenge-response based on symmetric-key crypto

- Unilateral authentication, timestamp-based
 - A to B: $\text{MAC}_K(t_A, B)$
- Unilateral authentication, nonce-based
 - B to A: r_B
 - A to B: $\text{MAC}_K(r_B, B)$
- Mutual authentication, nonce-based
 - B to A: r_B
 - A to B: $r_A, \text{MAC}_K(r_A, r_B, B)$
 - B to A: $\text{MAC}_K(r_B, r_A)$

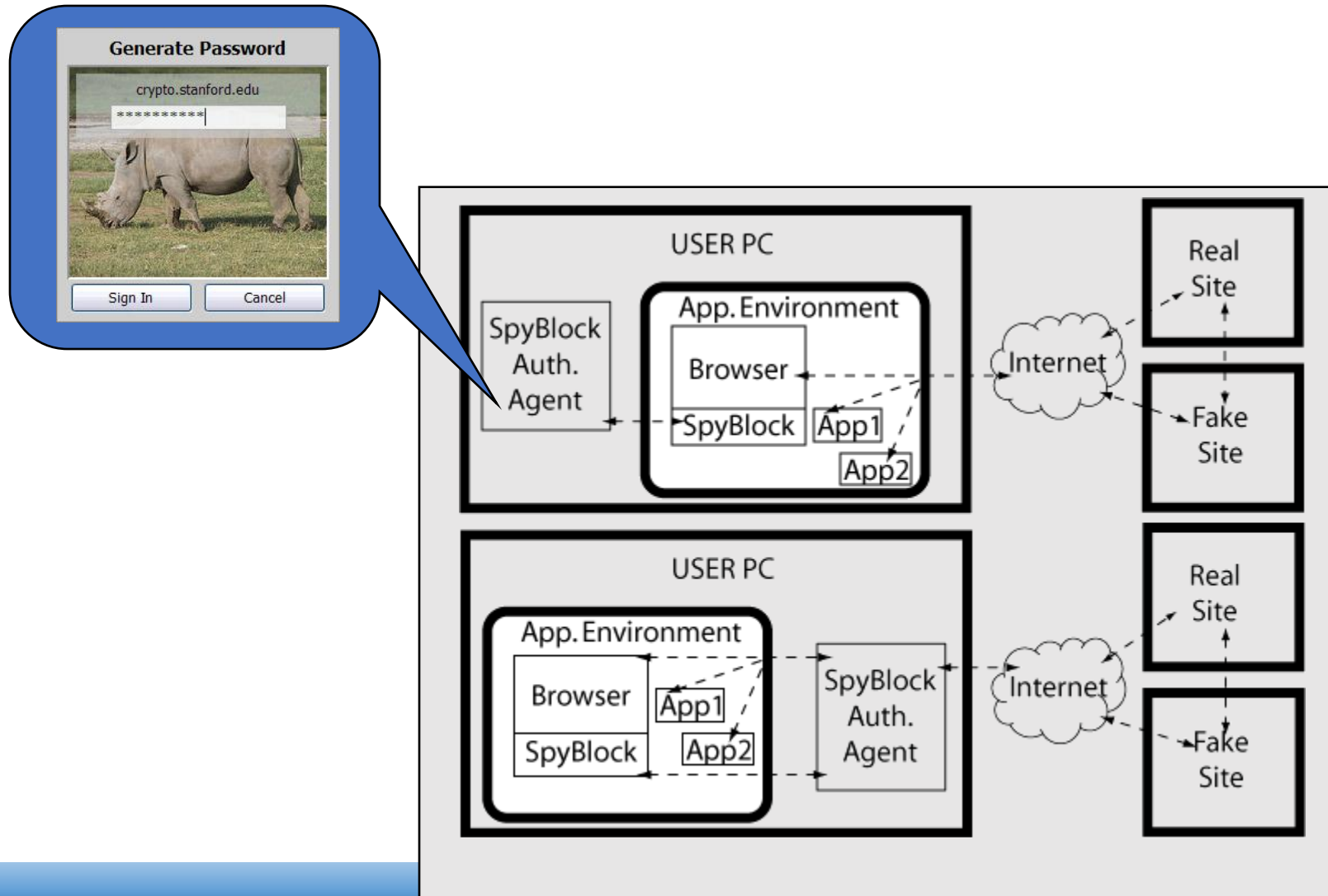
Other Defenses

- Alternatives to passwords
 - graphical passwords
- Go beyond passwords
 - security tokens
 - biometrics
 - 2-factor authentication
 - Uses two independent authentication methods
 - US Banks are required to use 2-factor authentication by end of 2006 for online banking
 - Out of band authentication: uses a channel other than the internet
 - E.g., phone

Open Problems

- Alternatives to passwords?
 - The secret should be easy to remember, difficult to guess, and easy to enter into the system.
- Better measure of password quality.
- Better ways to make user choose stronger passwords.
- Better ways to use other devices for authentication
- Effective 2-factored and/or out of band authentication for the Web
- Phishing defense

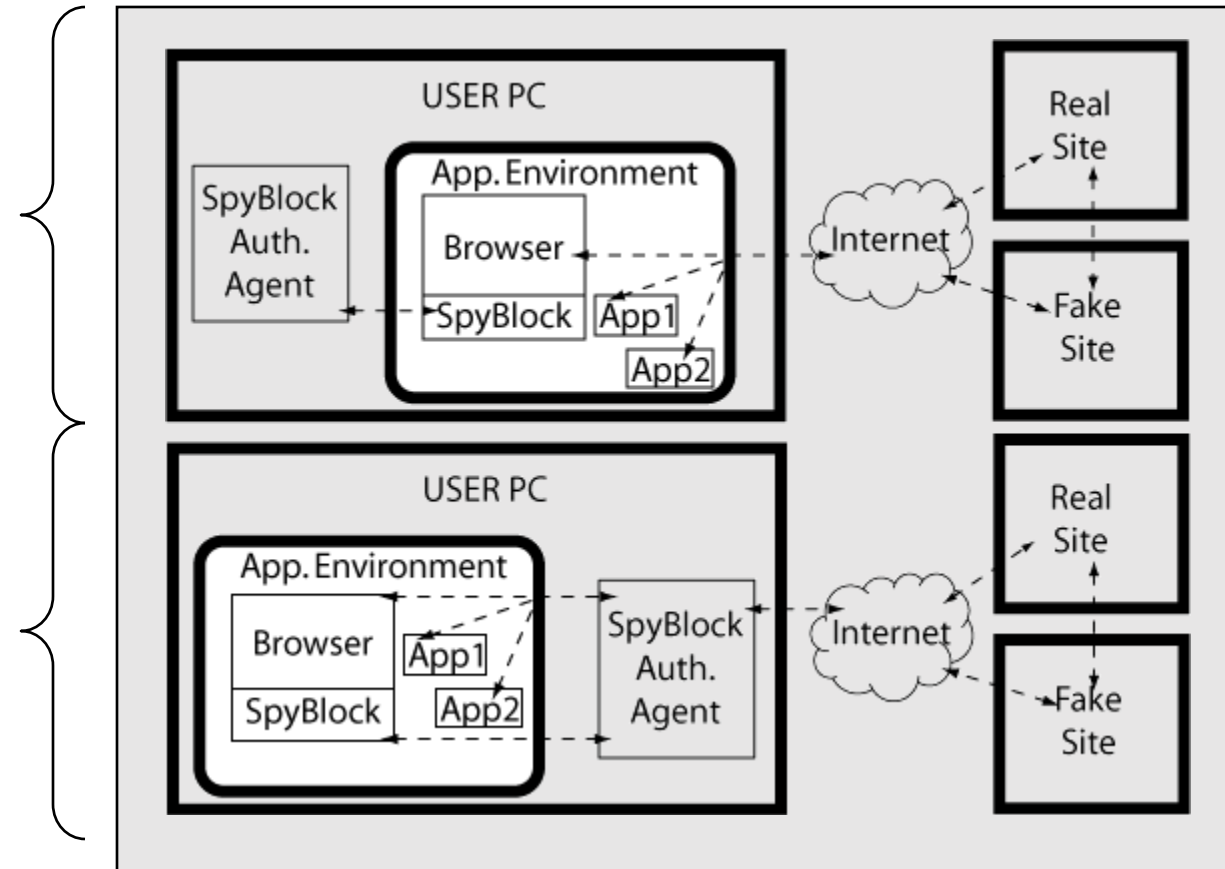
Defense: SpyBlock



Defense: SpyBlock

Authentication agent communicates through browser agent

Authentication agent communicates directly to web site



Password Storage

- What if a database is compromised?
- Dump all users passwords – in plaintext!!!
- <https://haveibeenpwned.com/>
 - Emails
 - Passwords

Hashing Example

- In Kali...

```
echo -n "password" | md5sum
```

- Some hashing algorithms...
 - MD5
 - Can fairly quickly compute collisions
 - SHA-1
 - <http://shattered.io>
 - SHA-2 family
 - SHA-256
 - Bcrypt
 - Specifically for hashing passwords
- Sample hashes: <http://openwall.info/wiki/john/sample-hashes>

Crunch

- Generates a wordlist from characters you specify (included in Kali)

```
root@kali:/usr/share/wordlists# crunch
crunch version 3.6

Crunch can create a wordlist based on criteria you specify. The output from crunch can be sent to the screen, file, or to another program.

Usage: crunch <min> <max> [options]
where min and max are numbers

Please refer to the man page for instructions and examples on how to use crunch
root@kali:/usr/share/wordlists# crunch 4 5 AB
Crunch will now generate the following amount of data: 272 bytes
0 MB
0 GB
0 TB
0 PB
Crunch will now generate the following number of lines: 48
AAAA
AAAB
AABA
AABB
```

Cewl

```
root@kali:/usr/share/wordlists# cewl --help
CeWL 5.1 Robin Wood (robin@digi.ninja) (http://digi.ninja)

Usage: cewl [OPTION] ... URL
    --help, -h: show help
    --keep, -k: keep the downloaded file
    --depth x, -d x: depth to spider to, default 2
    --min_word_length, -m: minimum word length, default 3
    --offsite, -o: let the spider visit other sites
    --write, -w file: write the output to the file
    --ua, -u user-agent: useragent to send
    --no-words, -n: don't output the wordlist
    --meta, -a include meta data
    --meta_file file: output file for meta data
    --email, -e include email addresses
```

- Included in Kali
- Creates wordlist from URL, reading words from pages

Hydra

- Online password cracker
- Can use wordlists or patters

```
root@kali:~/brute# hydra -l root -p password attack.samsclass.info http-get /brute0/
Hydra v7.4.2 (c)2012 by van Hauser/THC & David Maciejak - for legal purposes only

Hydra (http://www.thc.org/thc-hydra) starting at 2013-09-06 14:14:38
[DATA] 1 task, 1 server, 1 login try (l:1/p:1), ~1 try per task
[DATA] attacking service http-get on port 80
[80][www] host: 199.188.72.153 login: root password: password
1 of 1 target successfully completed, 1 valid password found
Hydra (http://www.thc.org/thc-hydra) finished at 2013-09-06 14:14:42
root@kali:~/brute#
```

Password Cracking – another option

- onlinehashcrack.com

**ONLINE HASH CRACK IS A PASSWORD RECOVERY SERVICE
ASSISTING PENTESTERS & SECURITY EXPERTS SINCE 2008**

Password/Hashes crack

YOUR HASHES (UP TO 10):

One hash per line

ALGORITHM:

Select hashtype..

EMAIL:

valid email for notification

SUBMIT

Wifi WPA(2) crack

UPLOAD YOUR CAPTURE FILE:

Choose File No file chosen

*.cap or *.pcap or *.hccapx

Max size : 20 Mb

Process all ESSID(s).

EMAIL:

Valid email for notification

SUBMIT

MS Office crack

UPLOAD YOUR OFFICE FILE:

Choose File No file chosen

*.doc or *.xls or *.ppt

Max size : 20 Mb

Encrypted Office 97-2003 files only

EMAIL:

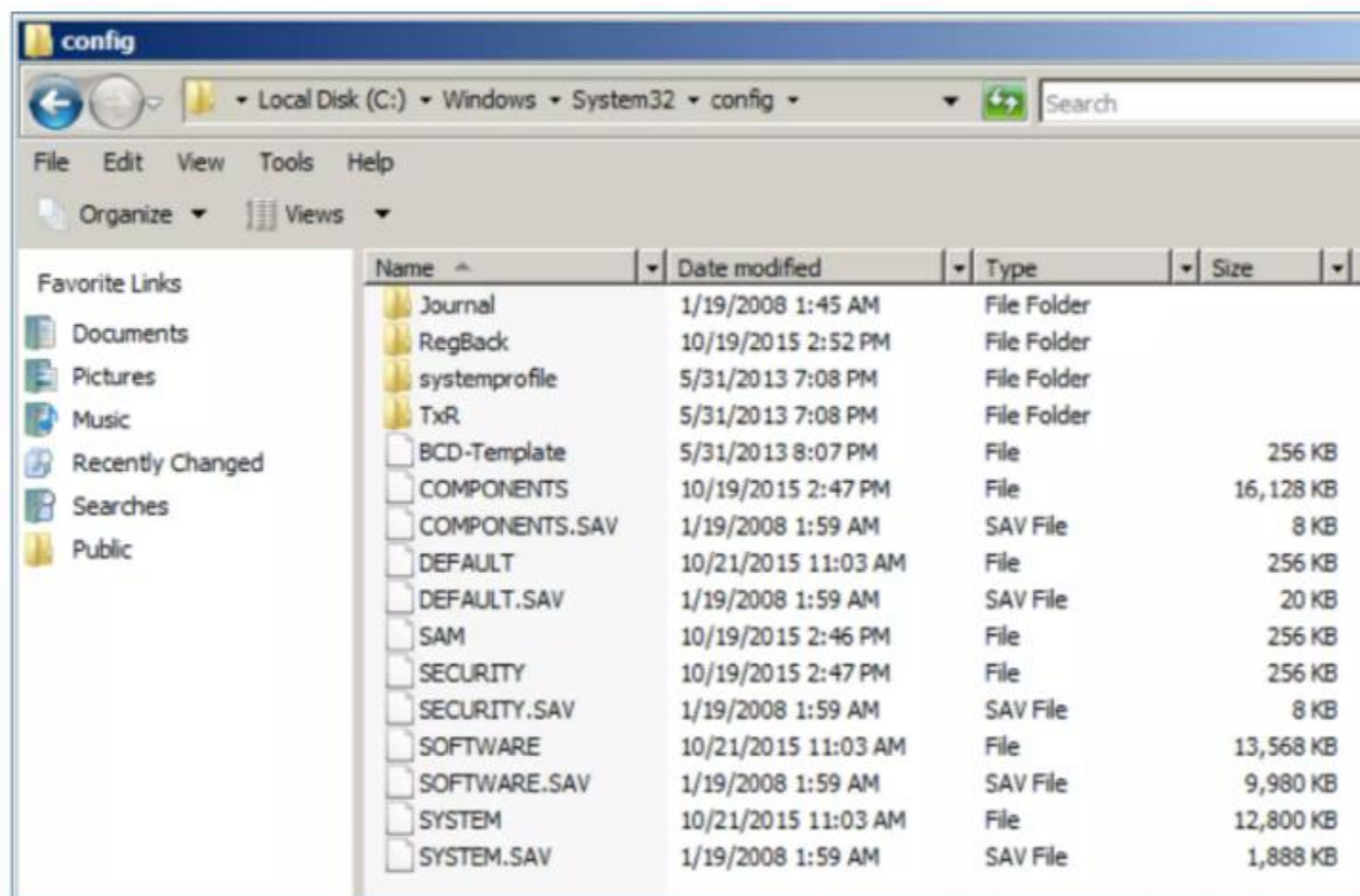
Valid email for notification

SUBMIT

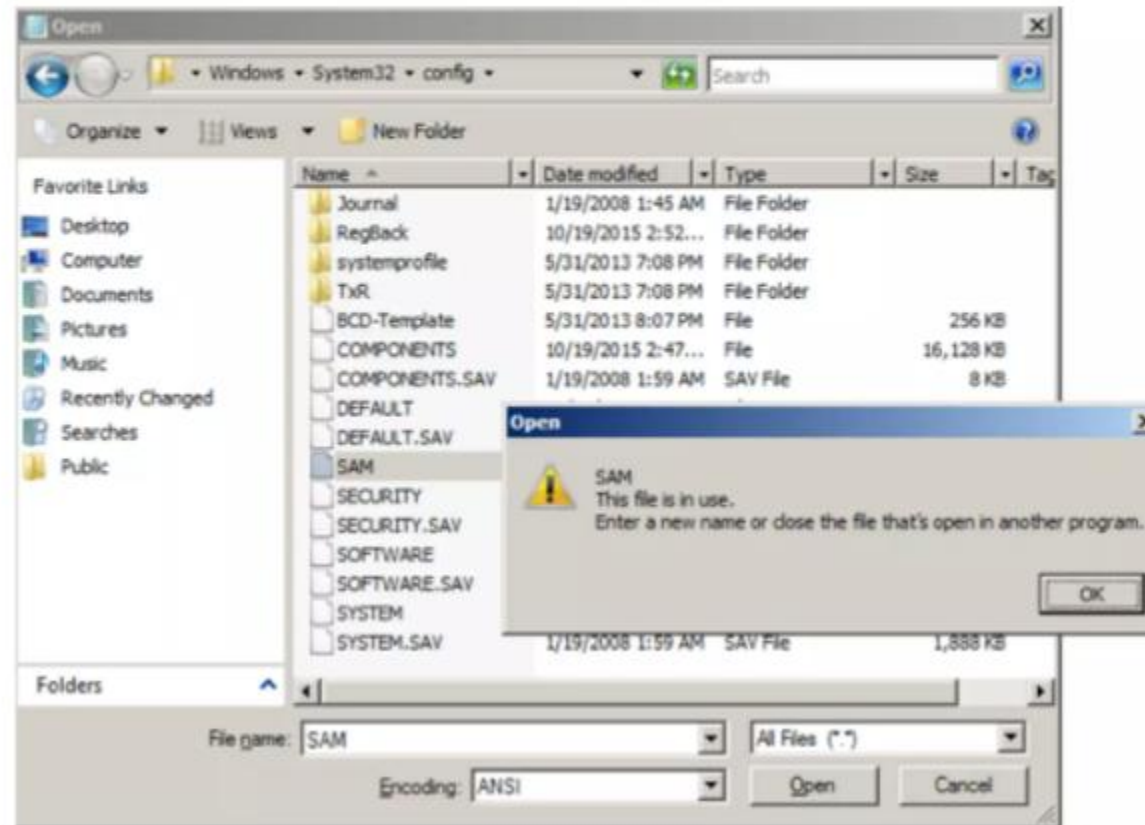
Getting the Hashes

- Most operating systems and Web services now hash passwords
 - Although some use plaintext, and most use weak hashing techniques
- Windows stores hashes in an encrypted C:\Windows\SAM file, but the key is available in the SYSTEM file

SAM and SYSTEM Files

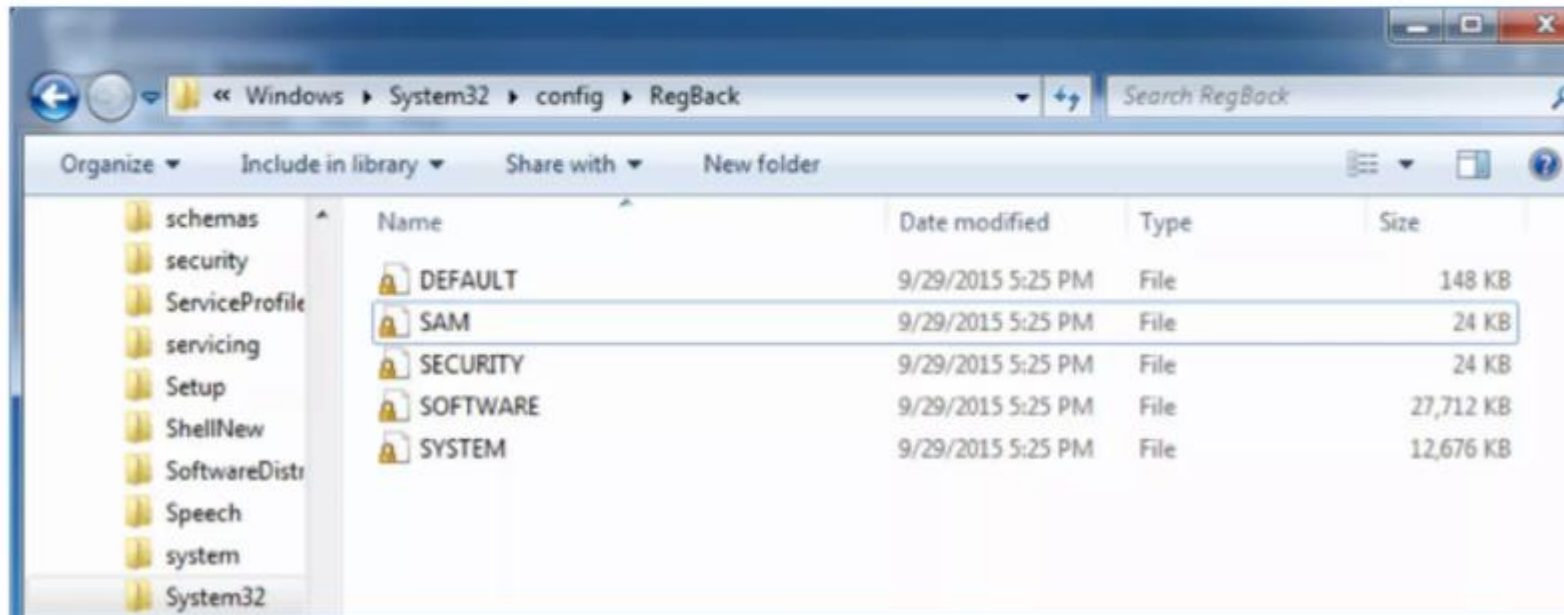


Unavailable when Windows is Running



Win 7 Backup Files

- Also unavailable when system is running
- Win XP had C:\Windows\Repair but it seems to be gone now



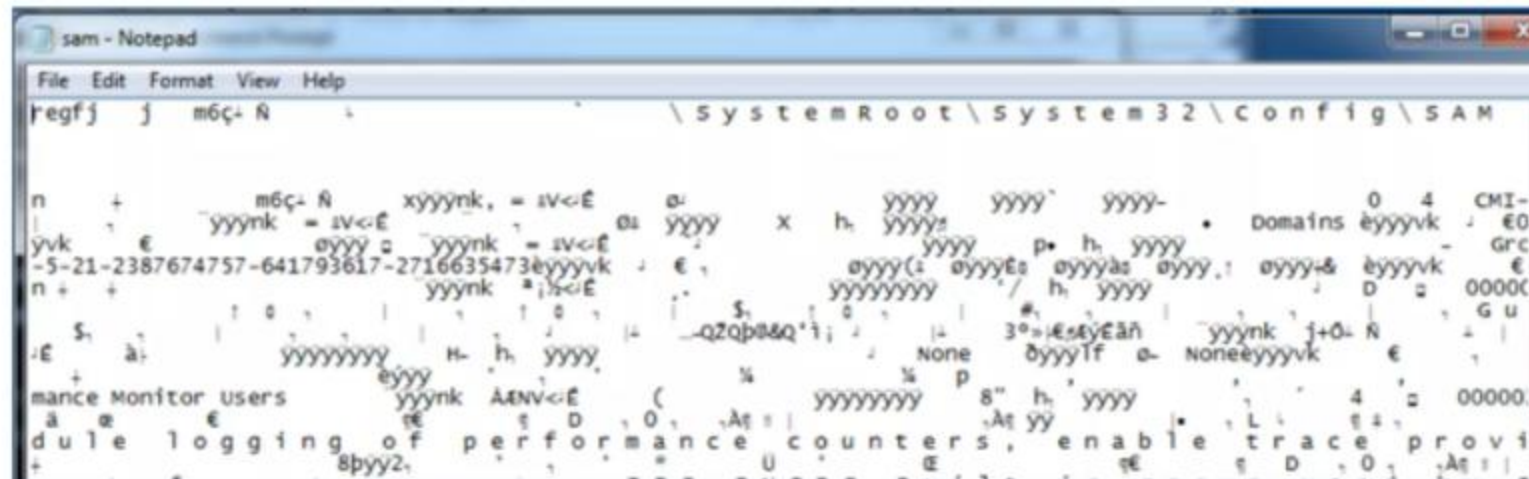
Reg.exe

- Works on Windows 7
 - Link Ch 8i

```
C:\>reg.exe save HKLM\SAM sam  
The operation completed successfully  
C:\>reg.exe save HKLM\SYSTEM sys  
The operation completed successfully
```

SAM is Encrypted

- 128-bit RC4



Key is in SYSTEM

- apt-get install bkhive FAILS on Kali 2
- Must install old versions of bkhive and samdump2 (link Ch 8l)

```
root@kali:~/124/ch9# bkhive system keyfile
bkhive 1.1.1 by Objectif Securite
http://www.objectif-securite.ch
original author: ncuomo@studenti.unina.it

Root Key : CMI-CreateHive{F10156BE-0E87-4EFB-969E-5DA29D131144}
Default ControlSet: 001
Bootkey: a8624e8beb93d2e7002c9eb240b53bdb
```

Key is in SYSTEM

- apt-get install bkhive FAILS on Kali 2
- Must install old versions of bkhive and samdump2 (link Ch 8l)

```
root@kali:~/124/ch9# bkhive system keyfile
bkhive 1.1.1 by Objectif Securite
http://www.objectif-securite.ch
original author: ncuomo@studenti.unina.it

Root Key : CMI-CreateHive{F10156BE-0E87-4EFB-969E-5DA29D131144}
Default ControlSet: 001
Bootkey: a8624e8beb93d2e7002c9eb240b53bdb
```

Thank You